Information Processing Letters 112 (2012) 423–426

Contents lists available at SciVerse ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Quadratic-time algorithm for a string constrained LCS problem

Sebastian Deorowicz

Silesian University of Technology, Institute of Informatics, Akademicka 16, Gliwice, Poland

ARTICLE INFO

Article history:
Received 15 July 2011
Received in revised form 9 January 2012
Accepted 13 February 2012
Available online 15 February 2012
Communicated by Ł. Kowalik

Keywords:
Algorithms
Sequence similarity
Longest common subsequence
Constrained longest common subsequence

ABSTRACT

The problem of finding a longest common subsequence of two main sequences with some constraint that must be a substring of the result (STR-IC-LCS) was formulated recently. It is a variant of the constrained longest common subsequence problem. As the known algorithms for the STR-IC-LCS problem are cubic-time, the presented quadratic-time algorithm is significantly faster.

1. Introduction

One of the most popular ways of measuring sequence similarity is computation of their longest common subsequence (LCS) [7], in which we are interested in a subsequence that is common to all sequences and has the maximal possible length. It is well known that for two sequences of length m and n an LCS can be found in $O(mn)$ time, which is a lower bound of time complexity in the comparison-based computing model for this problem [1]. In the more practical, RAM model of computations, the asymptotically fastest algorithm is the one by Masek and Paterson which runs in $O(mn/\log n)$ time for bounded and $O(mn \log\log n/\log n)$ time for unbounded alphabet [8].

One family of LCS-related problems considers one or more *constraining* sequences, such that (in some variants) must be included, or (in other variants) are forbidden as part of the resulting sequence [3,9]. The motivation for these generalizations came from bioinformatics in which some prior knowledge is often available and one can specify some restrictions on the result [9,5].

In this work, we consider the problem called STR-IC-LCS, introduced in [3], in which a constraining sequence of length r must be included as a substring of a common subsequence of two main sequences and the length of the result must be maximal. In [3] an $O(nmr)$-time algorithm was given for it. Farhana et al. [6] proposed finite-automata-based algorithms for the STR-IC-LCS, constrained longest common subsequence (CLCS), and two other problems defined by Chen and Chao [3]. The authors claim that the algorithms work in $O(r(m+n)+(m+n)\log(m+n))$ time in the worst case. It seems to be a breakthrough as it means also that the LCS problem could be solved in $O(n \log n)$ time. Unfortunately, the time complexity analysis are based on the claim from [2] that a directed acyclic subsequence graph (DASG) for two sequences of lengths m and n contains $O(m+n)$ states and can be built in $O((m+n)\log(m+n))$ time. As was shown by Crochemore et al. [4] this result was wrong and such a DASG contains $\Omega(mn)$ states in the worst case, so its construction time cannot be lower. Thus, the algorithms by Farhana et al. [6] work in $\Omega(mnr)$ time for the considered variants of the CLCS problem and $\Omega(mn)$ for the LCS problem. Moreover, these complexities are under assumption that the alphabet size is constant, otherwise they should be multiplied by its size.

In this paper, we propose the first quadratic-time algorithm for the STR-IC-LCS problem and show also further possible improvements of the time complexity. We also present how this algorithm can be extended to many main input sequences.

E-mail address: sebastian.deorowicz@polsl.pl.

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2012.02.007

The paper is organized as follows. In Section 2, some definitions are given and the problem is formally stated. Section 3 describes our algorithm. Extension to the case of many main sequences and some improvements of the algorithm are given in Section 4. The last section concludes.

2. Definitions

Let us have two main sequences $A = a_1a_2 \dots a_n$ and $B = b_1b_2 \dots b_m$ and one constraining sequence $P = p_1p_2 \dots p_r$. W.l.o.g. we can assume that $r \leqslant m \leqslant n$. Each sequence is composed of symbols from *alphabet* Σ of size σ. The *length* (or *size*) of any sequence X is the number of elements it is composed of and is denoted as $|X|$. A sequence $X^\star$ is a subsequence of X if it can be obtained from X by removing zero or more symbols. The LCS problem for A and B is to find a subsequence C of both A and B of the maximal possible length. The LCS length for A and B is denoted by $LLCS(A, B)$. A sequence β is a *substring* of X if $X = \alpha\beta\gamma$ for some, possibly empty, sequences α, β, γ. An *appearance* of sequence $X = x_1x_2 \dots x_{|X|}$ in sequence $Y = y_1y_2 \dots y_{|Y|}$, for any X and Y, starting at position j is a sequence of strictly increasing indexes $i_1, i_2, \dots, i_{|X|}$ such that $i_1 = j$, and $X = y_{i_1} \dots y_{i_{|X|}}$. A *compact* appearance of X in Y starting at position j is the appearance of the smallest last index, $i_{|X|}$. A *match* for sequences A and B is a pair (i, j) such that $a_i = b_j$. The total number of matches for A and B is denoted by d. It is obvious that $d \leqslant mn$.

The STR-IC-LCS problem for the main sequences A, B, and the constraining sequence P is to find a subsequence C of both A and B of the maximal possible length containing P as its substring. (In the CLCS problem, P must be a subsequence of C.)

3. The algorithm

The algorithm we propose is based on dynamic programming with some preprocessing. To show its correctness it is necessary to prove some lemmas.

Let $C = c_1c_2 \dots c_\ell$ be a longest common subsequence with substring constraint (STR-IC-LCS) for A, B, and P. Let also $I = (i_1, j_1), (i_2, j_2), \dots, (i_\ell, j_\ell)$ be a sequence of indexes of C symbols in A and B, i.e., $C = a_{i_1}a_{i_2} \dots a_{i_\ell}$ and $C = b_{j_1}b_{j_2} \dots b_{j_\ell}$. From the problem statement, there must exist such $q \in [1, \ell - r + 1]$ that $P = a_{i_q}a_{i_{q+1}} \dots a_{i_{q+r-1}}$ and $P = b_{j_q}b_{j_{q+1}} \dots b_{j_{q+r-1}}$.

Lemma 1. *Let $i'_q = i_q$ and for all $t \in [1, r-1]$, i'_{q+t} be the smallest possible, but larger than i'_{q+t-1}, index in A such that $a_{i_{q+t}} = a_{i'_{q+t}}$. The sequence of indexes $I' = (i_1, j_1), \dots, (i_{q-1}, j_{q-1}), (i'_q, j_q), (i'_{q+1}, j_{q+1}), \dots, (i'_{q+r-1}, j_{q+r-1}), (i_{q+r}, j_{q+r}), \dots, (i_\ell, j_\ell)$ defines a longest common subsequence of A and B with string constraint P equal C.*

Proof. From the definition of indexes i'_{q+t} it is obvious that they form an increasing sequence, since $i'_q = i_q$, and $i'_{q+r-1} \leqslant i_{q+r-1}$. The sequence $i'_q, \dots, i'_{q+r-1}$ is of course a compact appearance of P in A starting at i_q. Therefore, both components of I' pairs form increasing sequences and for any (i'_u, j_u), $a_{i'_u} = b_{j_u}$, so sequence I' defines an STR-IC-LCS C' equal to C. □

A similar lemma can be formulated for the j-th components of sequence I. Thus, it is easy to conclude that when looking for an STR-IC-LCS, instead of checking any common subsequences of A and B it suffices to check only such common subsequences that contain compact appearances of P both in A and B. (This is a direct consequence of the fact that $LLCS(X, Y) \leqslant LLCS(X, \alpha Y)$ for any sequence α.)

The number of different compact appearances of P in A and B will be denoted by d^A and d^B, respectively. It is easy to notice that $d^Ad^B \leqslant d$, since a pair (i, j) defines a compact appearance of P in A starting at i-th position and compact appearance of P in B starting at j-th position only for some matches.

The algorithm computing an STR-IC-LCS (Fig. 1) consists of three main stages. In the first stage both main sequences are preprocessed to determine, for each occurrence of the first symbol of P, the index of the last symbol of a compact appearance of P. In the second stage two DP matrices are computed: the forward one and the reverse one. The recurrence is exactly as for the LCS computation.

In the last stage the result is determined. To this end for each match (i, j) for A and B the ends (i', j') of compact appearances of P in A starting at i-th position and in B starting at j-th position are read. The length of an STR-IC-LCS containing these appearances of P is determined as a sum of the LCS length of prefixes of A and B ending at i-th and j-th positions, respectively, the LCS length of suffixes of A and B starting at i'-th and j'-th positions, respectively, and the constraint length. Since the first and last constraint symbol was summed twice, the final result is decreased by 2. According to the F and R matrices (containing the results of the classical dynamic programming recurrence computations for A and B and reversed A and B sequences, respectively), backtracking can be used to obtain the subsequence, not only its length.

Lemma 2. *The STR-IC-LCS algorithm* (*Fig.* 1) *correctly computes an STR-IC-LCS.*

Proof. The algorithm considers all pairs of compact appearances of P in A and B. Each such a pair divides the problem into two independent LCS length-computing subproblems. According to the precomputed F and R matrices it is easy to solve these subproblems (lines 18–20) in constant time. The length of an STR-IC-LCS must be a sum of the found lengths of LCSs and the constraint length subtracted by 2. □

Lemma 3. *The worst-case time complexity of the proposed algorithm is $O(mn)$.*

Proof. The preprocessing stage can be done in $O((m+n)r)$ worst-case time. The main stage consists of computation of two DP matrices which needs $O(mn)$ time. In the final stage, the DP matrix is traversed and for each match a constant number of operations is performed, so these stages

```
STR-IC-LCS(A, B, P)
   {Preprocessing}
 1 for i ← 1 to n do
 2     if a_i = p_1 then M^A[i] ← smallest q such that p_1 ... p_r is a subsequence of a_i ... a_q
 3 for j ← 1 to m do
 4     if b_i = p_1 then M^B[j] ← smallest q such that p_1 ... p_r is a subsequence of b_j ... b_q
   {Computation of forward and reverse DP matrices}
 5 for i ← 0 to n + 1 do F[i, 0] ← 0; R[i, m + 1] ← 0
 6 for j ← 0 to m + 1 do F[0, j] ← 0; R[n + 1, j] ← 0
 7 for i ← 1 to n do
 8     for j ← 1 to m do
 9         if a_i = b_j then F[i, j] = F[i − 1, j − 1] + 1
10         else F[i, j] = max(F[i − 1, j], F[i, j − 1])
11 for i ← n downto 1 do
12     for j ← m downto 1 do
13         if a_i = b_j then R[i, j] = R[i + 1, j + 1] + 1
14         else F[i, j] = max(R[i + 1, j], R[i, j + 1])
   {Determination of the result}
15 ℓ ← 0; i* ← 0; j* ← 0
16 for i ← 1 to n do
17     for j ← 1 to m do
18         if a_i = b_j and F[i, j] + R[M^A[i], M^B[j]] + r − 2 > ℓ then
19             ℓ ← F[i, j] + R[M^A[i], M^B[j]] + r − 2
20             i* ← i; j* ← j
21 Backtrack from (i*, j*) according to F and obtain S^1
22 Backtrack from (M^A[i*], M^b[j*]) according to R and obtain S^2
23 return ℓ and S^1 p_2 p_3 ... p_{r−1} S^2
```

Fig. 1. A pseudocode of the STR-IC-LCS computing algorithm for two main sequences and one constraining sequence.

consume $O(mn)$ time. Summing these up gives $O(mn)$ time. □

Lemma 4. *The space consumption of the algorithm is* $O(mn)$.

4. Improvements and extensions

If one is interested only in the STR-IC-LCS length, it is easy to notice that F and R matrices can be computed row-by-row which means that only $O(m)$ words are necessary for them. The values of F and R for matches of symbols equal to p_1 in F and p_r in R must be, however, stored explicitly, so the space for them is $O(d^\star)$ (where $d^\star = O(mn)$ is the number of such matches). This gives the total space $O(n + d^\star)$. If also the subsequence is requested, the cells for all matches must be stored to allow backtracking, so the space is $O(n+d)$ (in the worst case $d = O(mn)$).

As the only cells that are necessary to be stored explicitly are those for matches, the Hunt–Szymanski method [7] can be used to speed up the computation of F and R matrices if the number of matches is small. Therefore, the second stage can be completed in $O(d \log\log m + n)$ time if $\sigma = O(n)$ and $O(d \log\log n + n \log n)$ otherwise. The time complexity of the final stage is $O(d)$. Adding the time for the preprocessing we obtain the worst-case time complexities: $O(d \log\log m + nr)$ for $\sigma = O(n)$ and $O(d \log\log n + n(r + \log n))$ time otherwise.

The generalization of the LCS problem for many sequences is direct, but the time complexity of the exact algorithm computing the multidimensional DP matrix is $O(2^z n^z)$, where z is the number of sequences of length $O(n)$ each [7]. It is easy to notice that according to Lemma 1 the STR-IC-LCS problem generalizes in the same way and the worst-case time complexity is also $O(2^z n^z)$.

5. Conclusions

We investigated the STR-IC-LCS problem introduced recently. The fastest algorithms solving this problem known to date needed cubic time in case of two main and one constraining sequences. Our algorithm is faster, as its time complexity is only quadratic. Moreover, the algorithm uses an LCS-computation procedure as a component and any progresses in the LCS computation can improve the time complexities of the proposed method.

We also pointed out an irrecoverable flaw in [6], in which the algorithm of better than cubic time complexity was recently proposed, i.e., we show this algorithm is supercubic in the worst case.

Acknowledgements

The author thanks Szymon Grabowski for reading preliminary versions of the paper and suggesting improvements.

References

[1] A.V. Aho, D.S. Hirschberg, J.D. Ullman, Bounds on the complexity of the longest common subsequence problem, Journal of the ACM 23 (1) (1976) 1–12.
[2] R.A. Baeza-Yates, Searching subsequences, Theoretical Computer Science 78 (1991) 363–376.
[3] Y.-C. Chen, K.-M. Chao, On the generalized constrained longest common subsequence problems, Journal of Combinatorial Optimization 21 (2011) 383–392.
[4] M. Crochemore, B. Melichar, Z. Troníček, Directed acyclic subsequence graph—overview, Journal of Discrete Algorithms 1 (2003) 255–280.
[5] S. Deorowicz, Bit-parallel algorithm for the constrained longest common subsequence problem, Fundamenta Informaticae 99 (4) (2010) 409–433.
[6] E. Farhana, J. Ferdous, T. Moosa, M.S. Rahman, Finite automata based algorithm for the generalized constrained longest common subsequence problems, in: Proceedings of the 17th International Conference

on String Processing and Information Retrieval, SPIRE'10, in: LNCS, vol. 6393, Springer, 2010, pp. 243–249.
[7] D. Gusfield, Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.
[8] W.J. Masek, M.S. Paterson, A faster algorithm computing string edit distances, Journal of Computer System Science 20 (1) (1980) 18–31.
[9] Y.-T. Tsai, The constrained common subsequence problem, Information Processing Letters 88 (2003) 173–176.